



JavaScript & objects, ES6 extensions and TypeScript

Mahhek Tahir

What is javascript?



- A lightweight, interpreted programming language.
- Used for creating dynamic and interactive web content.
- Runs in the browser (client-side) and on servers (Node.js).
- **Example Uses:**
 - Form validations.
 - Animations.
 - Updating DOM dynamically.



Way to add

Method	How it is added	Example
Inline JavaScript	Written inside HTML tag	<pre><button onclick="alert('Hi')">Click</button></pre>
Embedded JavaScript	Written inside <code><script></code> tag in HTML	<pre><script>alert("Hello");</script></pre>
External JavaScript	Written in separate <code>.js</code> file	<pre><script src="app.js"></script></pre>

JavaScript's Reputation



- Everything is type sensitive, including function, class, and variable names.
- The scope of variables in blocks is not supported. This means variables declared inside a loop may be accessible outside of the loop, counter to what one would expect.
- There is a `===` operator, which tests not only for equality but type equivalence.
- **Null** and **undefined** are two distinctly different states for a variable. **Null** Means no value / empty value intentionally. **undefined** means Variable declared but not initialized
- Semicolons are not required, but are permitted (and encouraged).

Javascript Basic Syntax



Variables:

```
name = "John";
```

```
let name = "John";
```

```
const pi = 3.14;
```

```
var age = 25;
```

Data Types: String, Number, Boolean, Object, Array, Undefined, null

Operators: +, -, *, /, %, ++, --, ==, ===, !=, !==, >, <, >=, <=, &&, ||.

```
5 != "5"    // false    (type conversion happens)
```

```
5 !== "5"   // true     (no type conversion)
```

Data Types



// Numbers:

```
let length = 16;
```

```
let weight = 7.5;
```

// Strings:

```
let color = "Yellow";
```

```
let lastName = "Johnson";
```

// Booleans

```
let x = true;
```

```
let y = false;
```

// Object:

```
const person = {firstName:"John", lastName:"Doe"};
```

// Array object:

```
const cars = ["Saab", "Volvo", "BMW"];
```

```
// Date object: const date = new Date("2022-03-25");
```



Difference: var, let and const

var - Function scope (will print)

```
if (true) {  
    var x = 10;  
}  
console.log(x);
```

let - Block scope (ReferenceError: y is not defined)

```
if (true) {  
    let y = 20;  
}  
console.log(y);
```

Arithmetic operators



JavaScript performs implicit type coercion

String Coercion (Number → String)

When a number is added to a string, JavaScript converts the number into a string and performs **concatenation**. **`console.log(2 + "2");`** // "22"

Number Coercion (String → Number)

When a string contains a number and is used with mathematical operators (**-**, *****, **/**), JavaScript converts it into a number. **`console.log("10" - 5);`** // 5

JS Assignment operators



Operator	Example	Same As
=	<code>x = y</code>	<code>x = y</code>
<code>+=</code>	<code>x += y</code>	<code>x = x + y</code>
<code>-=</code>	<code>x -= y</code>	<code>x = x - y</code>
<code>*=</code>	<code>x *= y</code>	<code>x = x * y</code>
<code>/=</code>	<code>x /= y</code>	<code>x = x / y</code>
<code>%=</code>	<code>x %= y</code>	<code>x = x % y</code>
<code>**=</code>	<code>x **= y</code>	<code>x = x ** y</code>

`**` means power

JS Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator



String methods

Method / Property	What it does (easy words)	Example	Result
<code>length</code>	Counts characters	<code>"Hello".length</code>	<code>5</code>
<code>charAt()</code>	Gets character at position	<code>"Hello".charAt(1)</code>	<code>"e"</code>
<code>toUpperCase()</code>	Makes text CAPITAL	<code>"hi".toUpperCase()</code>	<code>"HI"</code>
<code>toLowerCase()</code>	Makes text small	<code>"HI".toLowerCase()</code>	<code>"hi"</code>
<code>at()</code>	Gets character (supports -ve index)	<code>"Hello".at(-1)</code>	<code>"o"</code>
<code>[]</code>	Gets character by index	<code>"Hello"[0]</code>	<code>"H"</code>
<code>slice()</code>	Cuts part of string	<code>"Hello".slice(1,4)</code>	<code>"ell"</code>
<code>substring()</code>	Cuts part (no -ve index)	<code>"Hello".substring(1,4)</code>	<code>"ell"</code>
<code>substr()</code>	Cuts using start + length (old)	<code>"Hello".substr(1,3)</code>	<code>"ell"</code>

Slice and at supports negative value whereas `[]` (undefined) and `substring` (makes it 0 or swap) doesn't

JavaScript String Properties & Methods

Property/Method	Description	Example
length	Returns the number of characters in the string.	"Hello".length ➔ 5
charAt(index)	Returns character at the given index.	"Hello".charAt(1) ➔ 'e'
toUpperCase()	Converts string to uppercase.	"hello".toUpperCase() ➔ 'HELLO'
toLowerCase()	Converts string to lowercase.	"HELLO".toLowerCase() ➔ 'hello'
indexOf(substring)	Finds the first index of a substring.	"Hello".indexOf('l') ➔ 2
slice(start, end)	Extracts a section of the string.	"Hello".slice(1, 4) ➔ 'ell'
split(separator)	Splits string into an array of substrings.	"A,B,C".split(',') ➔ ['A', 'B', 'C']
replace(find, new)	Replaces part of the string.	"Hello".replace('l', 'y') ➔ 'Hey'
trim()	Removes whitespace from both sides.	" Hi ".trim() ➔ 'Hi'
reverse()	Reverses the characters in an array.	['H', 'i'].reverse() ➔ ['i', 'H']

(Use with split/join for strings)



Number methods

Method	What it does (easy)	Example	Result
<code>toString()</code>	Number → text	<code>(10).toString()</code>	<code>"10"</code>
<code>toFixed(n)</code>	Rounds to n decimals	<code>(3.456).toFixed(2)</code>	<code>"3.46"</code>
<code>toExponential()</code>	Shows number in scientific form	<code>(1000).toExponential()</code>	<code>"1e+3"</code>
<code>parseInt()</code>	Converts text to whole number	<code>parseInt("25")</code>	<code>25</code>
<code>parseFloat()</code>	Converts text to decimal number	<code>parseFloat("3.14")</code>	<code>3.14</code>
<code>Number()</code>	Converts value to number	<code>Number("10")</code>	<code>10</code>
<code>isNaN()</code>	Checks if value is not a number	<code>isNaN("abc")</code>	<code>true</code>



Array Methods

Method / Property	What it does (very easy)	Example	Result
<code>length</code>	Counts items in array	<code>[1,2,3].length</code>	3
<code>toString()</code>	Array → text	<code>[1,2].toString()</code>	"1,2"
<code>at()</code>	Gets item by position	<code>[10,20,30].at(1)</code>	20
<code>join()</code>	Joins items with symbol	<code>[1,2,3].join("-")</code>	"1-2-3"
<code>pop()</code>	Removes last item	<code>[1,2,3].pop()</code>	3
<code>push()</code>	Adds item at end	<code>[1,2].push(3)</code>	3 (new length)
<code>shift()</code>	Removes first item	<code>[1,2,3].shift()</code>	1
<code>unshift()</code>	Adds item at start	<code>[2,3].unshift(1)</code>	3 (new length)
<code>delete</code>	Removes value (leaves empty)	<code>delete arr[1]</code>	empty slot
<code>concat()</code>	Joins two arrays	<code>[1,2].concat([3,4])</code>	[1,2,3,4]



JS Dates

- ▶ `const d = new Date();` //current date and time
- ▶ `const d = new Date("2022-03-25");`
- ▶ `new Date(year,month,day,hours,minutes,seconds,ms)`
- ▶ `Math.random();`



JS Objects

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"};
```

```
let firstName = person.firstName
```

```
let age = person["age"]
```

Add new property: `person.nationality = "English";`

Delete a property: `delete person.age;`

Important: `const` here means that

```
person = { firstName = "def" } // error
```

```
but person.firstName = "Ali" // allowed
```




Objects

```
var item = new Object();  
item.title = "HTML Specification";  
item.status = "available";  
item.isAvailable = function () {  
    return this.status == "available";  
}
```

```
alert(item.title); // HTML Specification  
alert(item.isAvailable()); // true
```



Using Object Constructors

```
function Item(t){  
  
    this.title = t;  
    this.status = "available";  
    this.isAvailable = function (){  
        return this.status == "available";  
    }  
  
}
```

```
var item1 = new Item("HTML Specification");  
alert(item1.title); // HTML Specification  
alert(item1.isAvailable()); // true
```

```
var item2 = new Item("Javascript Specification");  
alert(item2.title); // Javascript Specification  
alert(item2.isAvailable()); // true
```



Extending Objects

```
var item1 = new Item("HTML Specification");
alert(item1.title); // HTML Specification
alert(item1.isAvailable()); // true

item1.issue = function(){
  this.status = "issued";
}

alert( item1.isAvailable() ); // true

item1.issue();

alert( item1.isAvailable() ); // false
```



Constructors - recommended

Properties are defined in the constructor, while functions are defined on `Item.prototype`

```
function Item(t) {
  this.title = t;
  this.status = "available";
}

Item.prototype.isAvailable = function () {
  return this.status === "available";
};

Item.prototype.issue = function () {
  this.status = "issued";
};

Item.prototype.receive = function () {
  this.status = "available";
};

var item1 = new Item("HTML Specification");
var item2 = new Item("JavaScript Specification");

alert(item1.isAvailable()); // true
item1.issue();
alert(item1.isAvailable()); // false
item2.issue();
alert(item2.isAvailable()); // false
```



Inheritance

```
class Animal {  
    constructor(name) {  
        this.name = name;  
    }  
  
    speak() {  
        return this.name + " makes a sound";  
    }  
}  
  
class Dog extends Animal {  
    bark() {  
        return this.name + " barks";  
    }  
}  
  
const d2 = new Dog("Buddy");  
  
console.log(d2.speak()); // Output: Buddy makes a sound  
console.log(d2.bark()); // Output: Buddy barks
```



class based syntax

```
class Item{
constructor(t){this.title=t;this.status="available";}
isAvailable(){return this.status=="available";}
issue(){this.status="issued";}
receive(){this.status="available";}
}
```

```
class Book extends Item{
constructor(t,a){super(t);this.author=a;}
}
```

```
var book=new Book("00SC","Bertrand Meyer");
alert(book.title);           // 00SC
alert(book.author);          // Bertrand Meyer
alert(book.isAvailable());   // true
book.issue();
alert(book.isAvailable());   // false
book.receive();
alert(book.isAvailable());   // true
```

Class



Inside a class { ... }, you are **NOT allowed** to write:

- let
- var
- const
- normal statements

A class body can contain **ONLY: constructor, methods, class fields** (special syntax)

```
class Item {  
  abc = "def";    // ✓ class field  
  
  constructor(t) {  
    this.title = t;  
  }  
  
  setPrice(p) {  
    this.price = p;  
  }  
}
```

```
class Item {  
  getAbc() {  
    return "def";  
  }  
}
```

```
class Item {  
  constructor(t) {  
    this.title = t;  
    this.abc = "def";    // ✓ attribute  
  }  
  
  setPrice(p) {  
    this.price = p;  
  }  
}
```

This automatically becomes `this.abc`

JS Control Statements



If-else Statements:

```
if (age > 18 || cond) {  
    console.log("You are  
an adult.");  
} else {  
    console.log("You are a  
minor.");  
}
```

Loops:

```
for (let i = 0; i < 5;  
i++) {  
    console.log(i);  
}
```

Switch Case.

```
let x = "0";  
switch (x) {  
    case 0:  
        text = "Off";  
        break;  
    case 1:  
        text = "On";  
        break;  
    default:  
        text = "No value  
found";  
}
```




JS Loops

```
for (expression 1; expression 2; expression 3) {  
    // code block to be executed  
}
```

Expression 1 is executed (one time) before the execution of the code block.

Expression 2 defines the condition for executing the code block.

Expression 3 is executed (every time) after the code block has been executed.



JS Loops

```
const cars = ["BMW", "Volvo", "Saab", "Ford"];
```

```
let len = cars.length;
```

```
let text = "";
```

```
for (let i=0; i < len; i++) {  
  text += cars[i] + "<br>";  
}
```

BMW

Volvo

Saab

Ford



JS Loops

```
const array = ["one", "two", "three"];  
array.forEach(function (item, index)  
{  
    console.log(item, index);  
});
```

```
one 0  
two 1  
three 2
```



Adding an Event to a Button Later in JavaScript

```
<button id="myButton">Click Me</button>

<script>

let btn = document.getElementById("myButton");

btn.addEventListener("click", () => {
    console.log("Hello!");
});

</script>
```

JS Functions



What are Functions?

- Reusable blocks of code.

```
function greet(name) {  
    return `Hello, ` + name;  
}  
console.log(greet("Alice")); // Hello, Alice!
```

```
// Function is called, the return value will end up in x  
let x = myFunction(4, 3);
```

```
function myFunction(a, b) {  
    // Function returns the product of a and b  
    return a * b;  
}
```



Callbacks

A callback is a function passed as an argument to another function. This technique allows a function to call another function

```
function calculate(a, b, callback) {  
    let result = callback(a, b);  
    console.log("Result:", result);  
}  
  
function add(x, y) {  
    return x + y;  
}  
  
calculate(5, 3, add);
```



ES6 extensions

modern JavaScript features that make code shorter, cleaner, and easier to read.

let age = 20; let msg = `My age is \${age}`; //string literals

Feature	Meaning (easy)	Example
let	Block-scope variable	let x = 10;
const	Fixed variable	const PI = 3.14;
Arrow functions	Short function syntax	const add = (a,b) => a + b;
Template literals	Strings with <code>\${}</code>	Hello <code>\${name}</code>
Destructuring	Unpack values	let {a} = obj;
Default parameters	Default values in function	function f(x = 1){}
Classes	OOP style in JS	class User {}
Promises	Handle async work	new Promise((res, rej) => {})



Destructuring

The destructuring assignment syntax unpack object properties into variables:

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50  
};  
  
// Destructuring  
  
let {firstName, lastName, country = "US"} = person;
```




Typescripting

TypeScript is a **superset of JavaScript**

Developed by **Microsoft**

Adds **types** to JavaScript

Helps catch **errors before running code**



Types in TypeScript

Basic Types

```
let name: string = "Ali";  
let age: number = 20;  
let isStudent: boolean = true;
```

Arrays

```
let marks: number[] = [80, 90, 70];
```

Function with types

```
function greet(name: string, age: number): string {  
    return `Hello ${name}, you are ${age} years old`;  
}
```

// Variable with type

```
let username: string = "Ali";  
let age: number = 20;
```

// Function with types

```
function greet(name: string, age: number): string {  
    return `Hello ${name}, you are ${age} years old`;  
}
```

```
console.log(greet(username, age));
```



Correct use

```
// Correct types  
let price: number = 500;  
let product: string = "Book";  
  
function totalPrice(p: number, qty: number): number {  
    return p * qty;  
}  
  
console.log(totalPrice(price, 2)); // 1000
```



Incorrect use of typescript

```
// Wrong types
let price: number = 500;

// TypeScript error:
price = "five hundred"; // ✗ Error: Type 'string' is not assignable to type 'number'

function totalPrice(p: number, qty: number): number {
  return p * qty;
}

// Another error:
console.log(totalPrice(price, "2")); // ✗ Error: Argument of type 'string' is not assignable to type 'number'
```



Resources to Learn

Slide 15: Resources to Learn More

- Websites:
 - [MDN Web Docs](#).
 - [JavaScript.info](#).
- Platforms:
 - FreeCodeCamp, Codecademy, or W3Schools.

